

Aperiodic Scheduling Servers in Real-Time Systems

Fixed-Priority and Dynamic-Priority Server Mechanisms

Based on: G. C. Buttazzo, Hard Real-Time Computing Systems:
Predictable Scheduling Algorithms and Applications, 3rd ed., Springer, 2011

Real-Time Systems and HW/SW Co-Design Course

1. Introduction

Real-time systems commonly contain two types of tasks: periodic tasks with fixed activation intervals and known worst-case execution times, and aperiodic tasks that are event-driven with no fixed period. Managing both task types simultaneously presents a fundamental scheduling challenge.

Naively handling aperiodic tasks leads to two unsatisfactory extremes:

- Background scheduling: aperiodic tasks only execute in idle CPU slots, resulting in potentially unbounded response times in heavily loaded systems.
- Highest-priority interrupt: aperiodic tasks preempt all periodic work, breaking schedulability guarantees for hard deadlines.

The server abstraction resolves this dilemma. A server is a periodic task (C_s , T_s) that acts as a surrogate for aperiodic requests. From the scheduler's perspective it is an ordinary periodic task, preserving all schedulability analysis. From the aperiodic task's perspective it provides a dedicated, bounded CPU budget.

This paper surveys the major server mechanisms described in Buttazzo [1], progressing from the simplest fixed-priority approach (Polling Server) through increasingly capable designs (Deferrable Server, Sporadic Server, Slack Stealing) and concluding with the dynamic-priority Total Bandwidth Server.

2. Background Scheduling

Background scheduling is the simplest baseline: aperiodic tasks execute only during CPU idle time left by periodic tasks. Its sole advantage is zero impact on periodic schedulability. Its critical weakness is that aperiodic response time is unbounded — in a system with 80% periodic utilisation, aperiodic tasks compete for the remaining 20% and may wait arbitrarily long during high-load intervals.

Buttazzo uses background scheduling as the lower-bound reference against which all server designs are compared [1, Ch. 5].

3. Polling Server (PS)

3.1 Mechanism

The Polling Server is a periodic task (C_s , T_s) scheduled under Rate Monotonic Scheduling (RMS). At the start of each server period:

- If aperiodic requests are pending: serve them using budget C_s until either the budget is exhausted or the queue is empty.
- If no aperiodic requests are pending: the budget is immediately discarded and the server suspends until the next period.

This discard rule is the defining characteristic of PS and the source of both its strength (schedulability simplicity) and weakness (poor responsiveness).

3.2 Worst-Case Response Time

A request arriving just after the polling instant finds the capacity already discarded. It must wait up to T_s for the remainder of the current period, up to T_s for the next full period, plus its own execution time C_a :

$$R_a \leq 2 * T_s + C_a$$

3.3 Schedulability

Because PS behaves exactly like a periodic task, the standard Liu & Layland utilisation bound applies directly [2]:

$$U_s + \sum (C_i/T_i) \leq n * (2^{1/n} - 1)$$

where $U_s = C_s/T_s$ is the server utilisation.

4. Deferrable Server (DS)

4.1 Mechanism

The Deferrable Server changes exactly one rule compared to PS: unused capacity is preserved rather than discarded at the replenishment instant. The server remains armed throughout the period. When a request arrives, it is served immediately as long as budget remains.

4.2 Improved Response Time

Because a request arriving at any point in the period can be served immediately (budget permitting), the worst-case response time improves to:

$$R_a \leq T_s + C_a$$

This is half the PS bound, representing a significant practical improvement.

4.3 Schedulability Problem: Back-to-Back Execution

The preserved capacity creates a back-to-back execution hazard at period boundaries. If DS uses its full budget C_s at the end of period k , and replenishes immediately at the start of period $k+1$, it effectively executes $2C_s$ in a very short window. This burst can cause periodic tasks to miss deadlines even when total utilisation appears safe. Consequently, the standard RM bound does not apply to DS and a tighter schedulability condition is required [1, Ch. 5, pp. 133-138].

5. Sporadic Server (SS)

5.1 Motivation

Introduced by Sprunt, Sha, and Lehoczky (1989) [3], the Sporadic Server resolves the tension between PS and DS: PS offers simple schedulability but poor responsiveness; DS offers good responsiveness but breaks the RM bound. SS achieves both simultaneously.

5.2 The Three Rules

Rule 1 — Capacity consumption: SS capacity C_s is consumed only when the server is active and executing at a priority level above the current aperiodic task.

Rule 2 — Replenishment trigger: Replenishment is not at a fixed period boundary. It is triggered at the moment capacity begins to be consumed, scheduled exactly T_s later.

Rule 3 — Replenishment amount: The amount replenished equals exactly the amount consumed in that activation.

5.3 Why the Standard RM Bound Holds

Because replenishment always follows consumption by exactly T_s , the interference SS causes on periodic tasks is identical to that of a plain periodic task (C_s, T_s). The back-to-back burst that breaks DS cannot occur. The standard RM schedulability test therefore applies:

$$U_s + \text{SUM}(C_i/T_i) \leq n * (2^{(1/n)} - 1)$$

SS achieves DS-level response time ($R_a \leq T_s + C_a$) with PS-level schedulability simplicity.

6. Slack Stealing

6.1 Concept

Slack Stealing dynamically computes the system slack at each scheduling point — the amount of CPU time that can safely be given to aperiodic tasks without causing any periodic deadline miss. The slack for task T_i at time t is:

$$\text{slack}_i(t) = d_i - t - \text{remaining_execution_time}(i)$$

Whenever total system slack is positive, the CPU is immediately available for aperiodic service.

6.2 Optimality and Cost

Slack Stealing is optimal among fixed-priority approaches — no other fixed-priority method can serve aperiodic tasks faster without risking a deadline miss. Its critical weakness is $O(n)$ runtime overhead per aperiodic arrival, requiring exact WCET knowledge of all periodic tasks. It serves primarily as a theoretical performance benchmark [1, Ch. 5, pp. 149-152].

7. Non-Existence of Optimal Fixed-Priority Servers

Buttazzo proves a fundamental impossibility result [1, Ch. 5, pp. 153-154]: no fixed-priority server can simultaneously minimise aperiodic response time (match Slack Stealing) and preserve the standard RM utilisation bound (match PS/SS). This is a proven structural limitation, not a gap to be closed. The trade-off between responsiveness and schedulability simplicity is inherent to fixed-priority scheduling, motivating the shift to dynamic-priority EDF-based servers.

8. Total Bandwidth Server (TBS)

8.1 EDF as the Enabling Policy

Earliest Deadline First (EDF) scheduling removes the fixed-priority limitation. Under EDF, CPU utilisation can reach 100% while guaranteeing all deadlines, compared to the approximately 69.3% ceiling of RMS.

8.2 Deadline Assignment Rule

Introduced by Spuri and Buttazzo (1994) [4], TBS assigns each incoming aperiodic request a deadline that ensures total aperiodic utilisation never exceeds reserved bandwidth U_s . When request j arrives at time r_j with execution time C_j :

$$d_j = \max(r_j, d_{\{j-1\}}) + C_j / U_s$$

where $d_{\{j-1\}}$ is the deadline of the previous aperiodic request.

8.3 Schedulability Condition

The system is schedulable under EDF if:

$$U_p + U_s \leq 1$$

where $U_p = \text{SUM}(C_i/T_i)$ is total periodic utilisation. This is the full EDF bound.

8.4 Numerical Example

Given $U_s = 0.25$ and two aperiodic requests:

- Request 1: $r_1 = 0$, $C_1 = 2$ ms $\Rightarrow d_1 = 0 + 2/0.25 = 8$
- Request 2: $r_2 = 1$, $C_2 = 1$ ms $\Rightarrow d_2 = \max(1, 8) + 1/0.25 = 12$

Both requests enter the EDF queue and are scheduled transparently alongside all periodic tasks.

9. Comparative Analysis

Table 1 summarises the key properties of all server mechanisms discussed.

Server	Unused Capacity	Worst-Case Response	RM Bound Applies	Implementation
Polling Server (PS)	Discarded	$2T_s + C_a$	Yes	Simple
Deferrable Server (DS)	Preserved	$T_s + C_a$	No (tighter needed)	Simple
Sporadic Server (SS)	Preserved (consumption-triggered)	$T_s + C_a$	Yes	Moderate
Slack Stealing	N/A (dynamic)	Optimal	Yes	High overhead
Total Bandwidth Server (TBS)	N/A (EDF-based)	Near-optimal	N/A (EDF bound)	Simple

Table 2 ranks the servers by aperiodic response time performance (Buttazzo [1, Ch. 5.9]).

Rank	Aperiodic Response Time	Schedulability Simplicity
------	-------------------------	---------------------------

1st	Slack Stealing	PS = SS = TBS
2nd	TBS (near-optimal)	DS
3rd	SS ~ DS	Priority Exchange
4th	PS	—
5th	Background	—

10. Conclusion

The evolution from Polling Server to Total Bandwidth Server reflects a progressive resolution of the core tension in hybrid task scheduling: how to serve aperiodic requests quickly without sacrificing schedulability guarantees for periodic tasks.

The Polling Server established the server abstraction but suffered from poor responsiveness due to its discard rule. The Deferrable Server improved responsiveness at the cost of schedulability simplicity. The Sporadic Server achieved both goals within fixed-priority scheduling. Slack Stealing proved optimal but impractical. Buttazzo's non-existence theorem then demonstrated that fixed-priority servers face an inherent ceiling, motivating the shift to EDF.

The Total Bandwidth Server, operating under EDF, resolves what fixed-priority servers structurally cannot: near-optimal aperiodic responsiveness with a clean schedulability condition up to full CPU utilisation, using a simple deadline assignment rule.

For practical system design, the Sporadic Server is the recommended choice under fixed-priority scheduling, while the Total Bandwidth Server is preferred when EDF is available.

References

- [1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York: Springer, 2011. Ch. 5 (Fixed-Priority Servers, pp. 121-156), Ch. 6 (Dynamic Priority Servers).
- [2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46-61, Jan. 1973.
- [3] B. Sprunt, L. Sha, and J. Lehoczky, "Aperiodic task scheduling for hard real-time systems," *Real-Time Systems*, vol. 1, no. 1, pp. 27-60, Jun. 1989.
- [4] M. Spuri and G. C. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179-210, Mar. 1996.
- [5] L. Sha, R. Rajkumar, and J. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175-1185, Sep. 1990.